System Design

Architecture

Layered Architecture – keeps the business rules apart from the console and from SQLite. That makes the late-penalty and moderation rules testable and keeps all SQL in one layer.

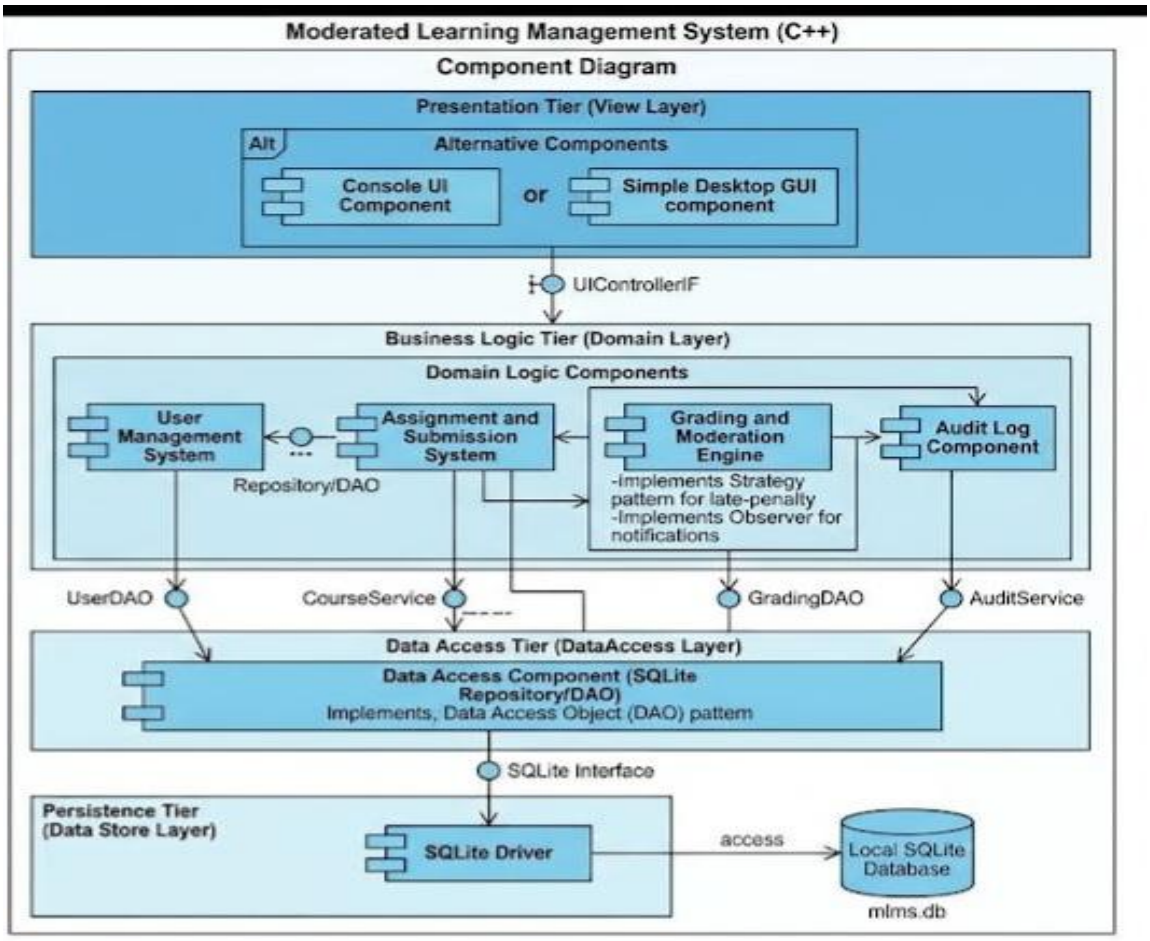
The application is structured into four distinct, hierarchically organized logical layers:

- **Presentation / UI layer:** Manages user interaction via a console interface or a lightweight desktop GUI. It accepts user inputs for authentication, submissions and grading, rendering responses securely based on the active user session.
- **Business Logic / Application Layer:** Encapsulates core domain rules, including role-based access checks, late penalty calculations (e.g. deducting 10% per late day up to 3 days), assignment extension validation, submission lifecycles, and audit logging
- **Data Access Layet / Repository Layer:** Acts as an abstraction barrier between domain logic and persistence. It executes parameterized queries and handles object relational mapping without exposing raw database statements to the business layer.
- **Persistence Layer:** A local SQLite database storing relational data for users, roles, courses, enrollments, assignments, submissions and audit logs.

Rejected alternatives:

- **Model-View-Controller:** separates several views from one model, but this system has one console view. In a console program the controllers tend to absorb the business rules (penalty moderation), which makes them hard to unit test and puts the coverage target at risk. The layered style keeps the rules in services and the domain, and the UI stays thin. Trade-off: layering adds forwarding code between layers, which the thin services keep small.
- **Client-server with a network database:** The system requirements explicitly mandate that the software must function fully offline on local school computers without relying on stable internet access. A distributed client-server model would introduce unnecessary network dependencies, deployment complexity, and security overhead for a single-machine local desktop environment.

1. Component Diagram



Component Descriptions and Responsibilities

- **Presentation Tier Components:**
  - **UI / Console View:** Captures user commands, renders menus tailored to the active user role (Admin, Lecturer, Head of Department, Student), and presents validation messages or feedback confirmation.
- **Business Logic Tier Components:**
  - **Authentication and security Manager:** Manages credential verification, password hashing verification (salted hashes), and session token creation.
  - **User and Role Controller:** Enforces strict role-based access control across operations, ensuring users can only execute authorized task.
  - **Courses and Assignment Manager:** Handles courses creation, student enrollment, assignment configuration, deadlines and individual/class extensions.

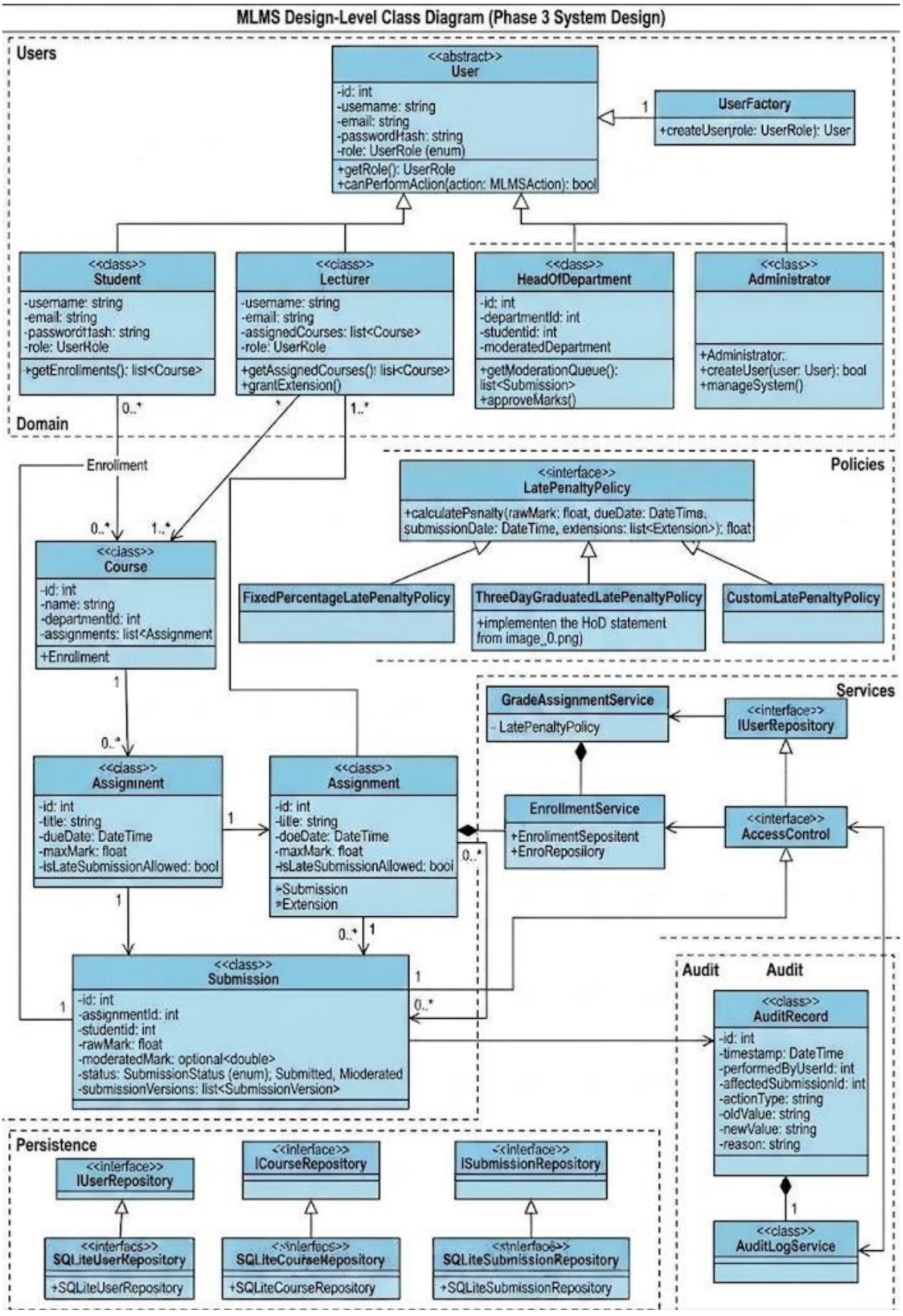
- **Submission and Grading Engine:** Coordinates the submission lifecycle (submitted, late, marked, moderated, released), applies late-penalty business rules, and manages lecturer grading workflows alongside Head of Department moderation
- **Audit and Logging Service:** Captures all changes made to student marks (recording who changed it, timestamp, old value, new value and reason).

- **Data Access Tier Components:**
  - **SQLite Repository / DAO Layer:** Provides clean data-access interfaces, maps domain entities to database tables, and executes parameterized SQL statements to safeguard against injection vulnerabilities.
- **Persistence Tier Components:**
  - **Local SQLite Database:** A self-containing, file-based relational database providing durable storage for offline usage.

2. Traceability to Requirements

- **Offline Capabilities:** The localized layered design combined with the SQLite persistence tiers satisfies **NFR-02** (operates without internet connectivity)
- **Role Enforcement:** Separated security and controller components ensure strict compliance with **FR-02, FR-03,** and **NRF-06.**
- **Auditability:** Dedicated audit logging components guarantees 100% trace capture of grade modifications to satisfy **FR-13** and **NRF-05**

Class Diagram


MLMS Design-Level Class Diagram (Phase 3 System Design)
Users
<<abstract>>
User
-id: int
-username: string
-email: string
-passwordHash: string
-role: UserRole (enum)
+getRole(): UserRole
+canPerformAction(action: MLMSAction): bool
1
UserFactory
+createUser(role: UserRole): User
<<class>>
Student
-username: string
-email: string
-passwordHash: string
-role: UserRole
+getEnrollments(): list<Course>
<<class>>
Lecturer
-username: string
-email: string
-assignedCourses: list<Course>
-role: UserRole
+getAssignedCourses(): list<Course>
+grantExtension()
<<class>>
HeadOfDepartment
-id: int
-departmentId: int
-studentid: int
-moderatedDepartment
+getModerationQueue(): list<Submission>
+approveMarks()
<<class>>
Administrator
+Administrator:
+createUser(user: User): bool
+manageSystem()
Domain
0..*
1..*
Enrollment
Policies
<<interface>>
LatePenaltyPolicy
+calculatePenalty(rawMark: float, dueDate: DateTime, submissionDate: DateTime, extensions: list<Extension>): float
FixedPercentageLatePenaltyPolicy
ThreeDayGraduatedLatePenaltyPolicy
+implementen the HoD statement from image_0.png)
CustomLatePenaltyPolicy
<<class>>
Course
-id: int
-name: string
-departmentId: int
-assignments: list<Assignment
+Enrollment
Services
GradeAssignmentService
- LatePenaltyPolicy
<<interface>>
IUserRepository
<<class>>
Assignment
-id: int
-title: string
-dueDate: DateTime
-maxMark: float
-isLateSubmissionAllowed: bool
<<class>>
Assignment
-id: int
-title: string
-doeDate: DateTime
-maxMark: float
-isLateSubmissionAllowed: bool
+Submission
+Extension
EnrollmentService
+EnrollmentSepositent
+EnroRepository
<<interface>>
AccessControl
<<class>>
Submission
-id: int
-assignmentId: int
-studentid: int
-rawMark: float
-moderatedMark: optional<double>
-status: SubmissionStatus (enum); Submitted, Mioderated
-submissionVersions: list<SubmissionVersion>
Audit
Audit
<<class>>
AuditRecord
-id: int
-timestamp: DateTime
-performedByUserId: int
-affectedSubmissionId: int
-actionType: string
-oldValue: string
-newValue: string
-reason: string
<<class>>
AuditLogService
Persistence
<<interface>>
IUserRepository
<<interface>>
ICourseRepository
<<interface>>
ISubmissionRepository
<<interface>>
SQLiteUserRepository
+SQLiteUserRepository
<<interface>>
SQLiteCourseRepository
+SQLiteCourseRepository
<<interface>>
SQLiteSubmissionRepository
+SQLiteSubmissionRepository

- **Class Diagram Structure and Design Explanation**
  - **User Hierarchy and Factor**
    - *<<abstract>> User* defines core attributes(*UserId, username, email, passwordHash, role*) and common methods (*getRole(), canPerformAction()*).
    - Concrete subclasses (*Student, Lecturer, HeadOfDepartment, Administrator*) inherit from User.
    - *UserFactory* implements the Factory Pattern to dynamically instantiate the correct user subclass based on role enumeration
  - **Academic Structure:**
    - A *Course* contains a collection of *Assignment* objects (*0..**)
    - An *Assignment* links to student *submission* attempts.
    - *Submission* tracks *rawMarks,* an optional *moderatedMark: double,* and status enums (*Submitted, Moderated, ReturnedForCorrection*).
  - **Policies and Services**
    - *LatePenaltyPolicy* is an interface implemented by concrete classes like *ThreeDayGraduatedLatePenaltyPolicy* (which encodes the Head of Department's rule: deducting 10% per day for up to three days, after which it receives zero)
    - *GradeAssignmentService* and *EnrollmentService* coordinbate domain operations
  - **Audit Logging:**
    - *AuditLogService* manages compliance recording, interacting with *AuditRecord* to store timestamps, performed user ID's and specific reasons for mark modifications.
  - **Persistence Layer(Repositories/DAOs):**
    - Generic and entity-specific interfaces (*IRepository<T>, IUserRepository, ICourseRepository, ISubmissionRepository*) decouple business logic from the database, implemented by concrete SQLite classes (*SQLiteUserRepositoty, etc*) using parameterized queries.

- **Design Patterns Justification**

1. **Factory Pattern (***UserFactory***):**

- *Choice:* Chosen for user account creation. Because the system handles four distinct user types (*Student, Lecturer, HeadOfDepartment, Administrator*), centralizing instantiation logic inside a factory prevents client code from tightly coupling to concrete subclass constructors and simplifies future role additions.

2. **Strategy Pattern (***LatePenaltyPolicy***):**
    - *Choice:* Chosen for calculating late submission penalties. Different assignments or future institutional policies might require custom deduction rules (e.g., flat percentages vs. graduated daily penalties). The Strategy pattern allows penalty algorithms to be swapped dynamically at runtime without modifying the core grading engine.
3. **Repository / DAO Pattern (***IRepository<T>* **& SQLite DAOs):**
    - *Choice:* Chosen to abstract data persistence. By forcing business logic to interact with repository interfaces rather than raw SQL statements, the design ensures the application logic remains completely independent of SQLite, satisfying the offline database requirement while facilitating unit testing with mock repositories.
4. **Observer Pattern (Implicitly via** *AuditLogService***):**
    - *Choice:* Chosen for audit trail notifications. Whenever a mark is modified by a lecturer or head of department, the audit logging service observes the state change and automatically records a persistent *AuditRecord* containing who changed it, when, and why.

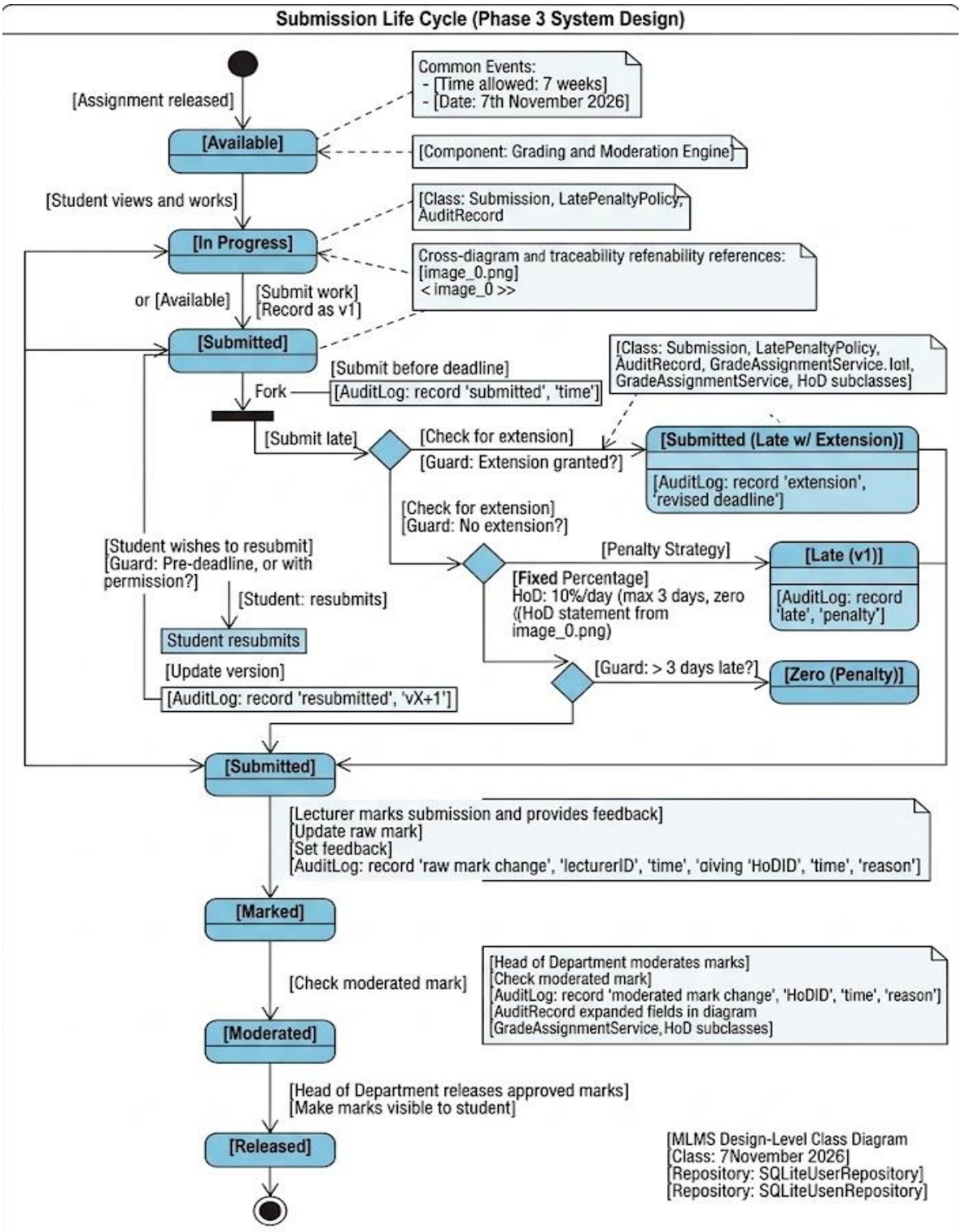
➢ **Compliance with SOLID Principles**

- **Single Responsibility Principle (SRP):**
    - Each class has a single, well-defined responsibility. For example, *GradeAssignmentService* handles grade calculations, *AuditLogService* manages audit records, and *SQLiteSubmissionRepository* handles database I/O. UI rendering and business logic are entirely separated.

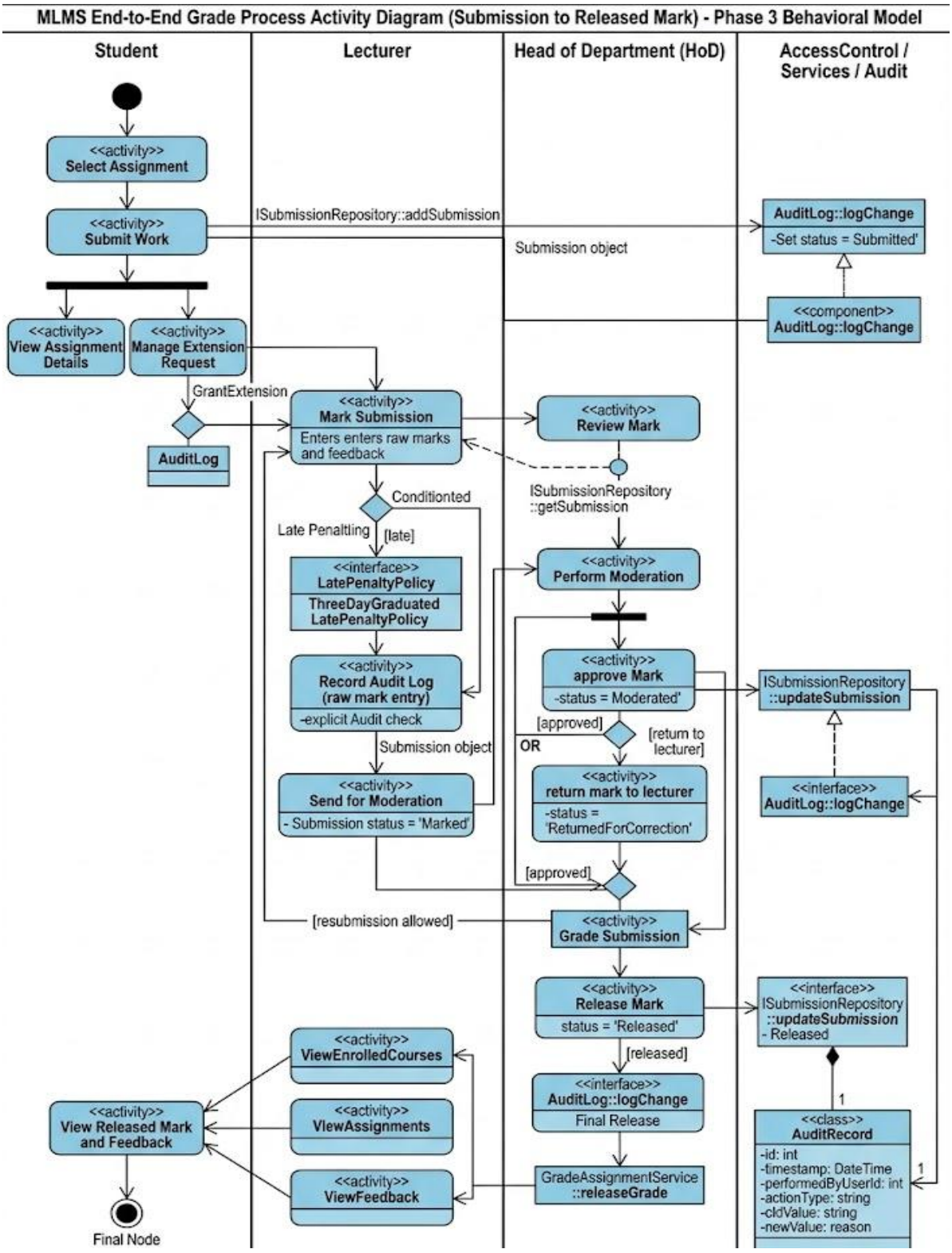
- **Open/Closed Principle (OCP):**
    - The system is open for extension but closed for modification. New penalty rules can be introduced by creating a new class implementing *LatePenaltyPolicy* without altering existing grading code. Similarly, adding a new user role requires extending *User* without breaking existing subclasses.
- **Liskov Substitution Principle (LSP):**
    - All concrete user subclasses (*Student, Lecturer, HeadOfDepartment, Administrator*) can substitute for the base *<<abstract>> User* class seamlessly. Similarly, any concrete repository can replace *IRepository<T>* without crashing application workflows.
- **Interface Segregation Principle (ISP):**
    - Interfaces are split cleanly by domain entity (*IUserRepository, ICourseRepository, ISubmissionRepository*) rather than forcing classes to implement bloated, monolithic data-access contracts.
- **Dependency Inversion Principle (DIP):**
    - High-level business modules (like *GradeAssignmentService*) depend upon abstractions (*LatePenaltyPolicy, ISubmissionRepository*) rather than concrete database drivers or low-level implementation details, ensuring loose coupling and testability.

Behavioral Models

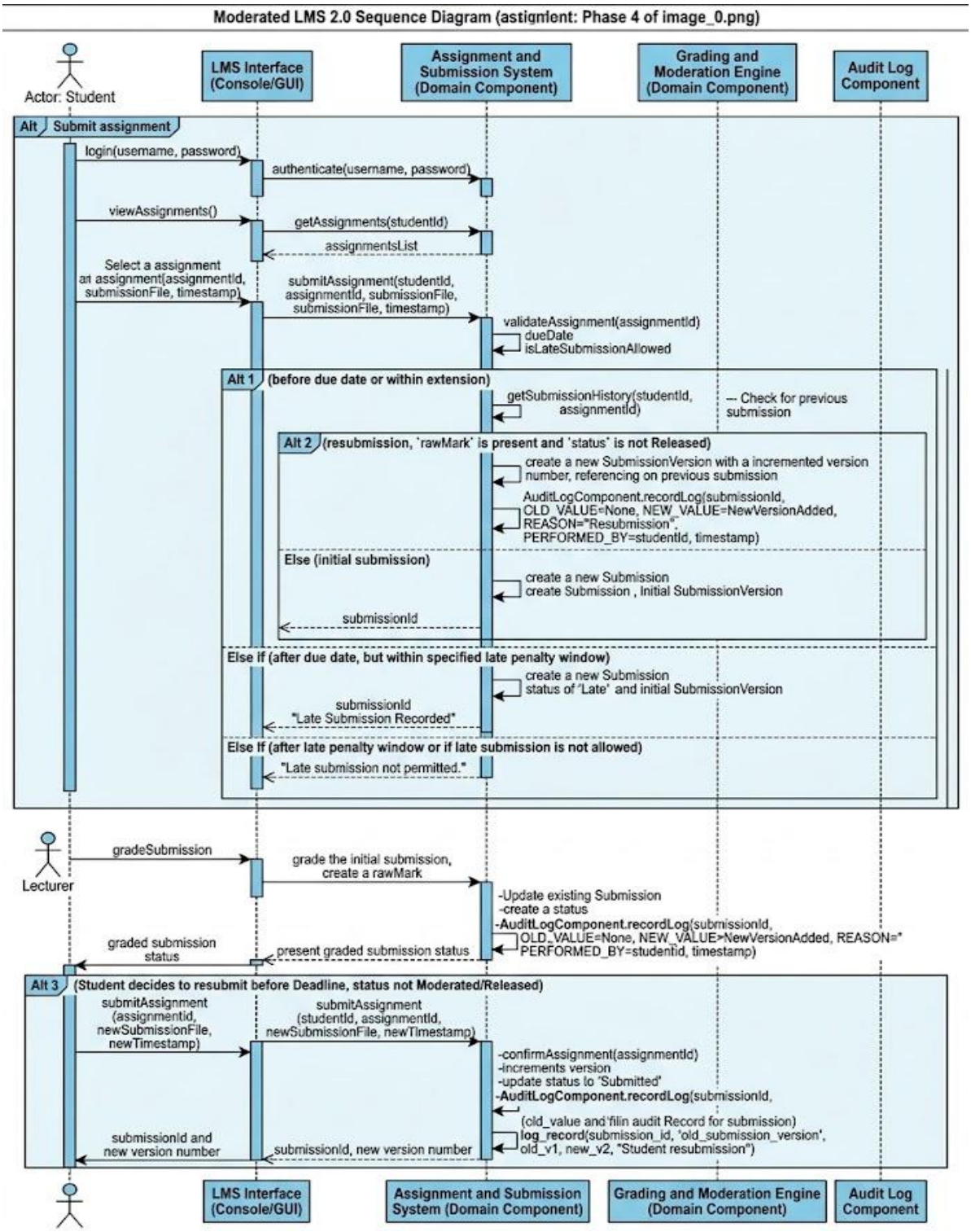
i. State Machine Diagram for the Submission Life Cycle



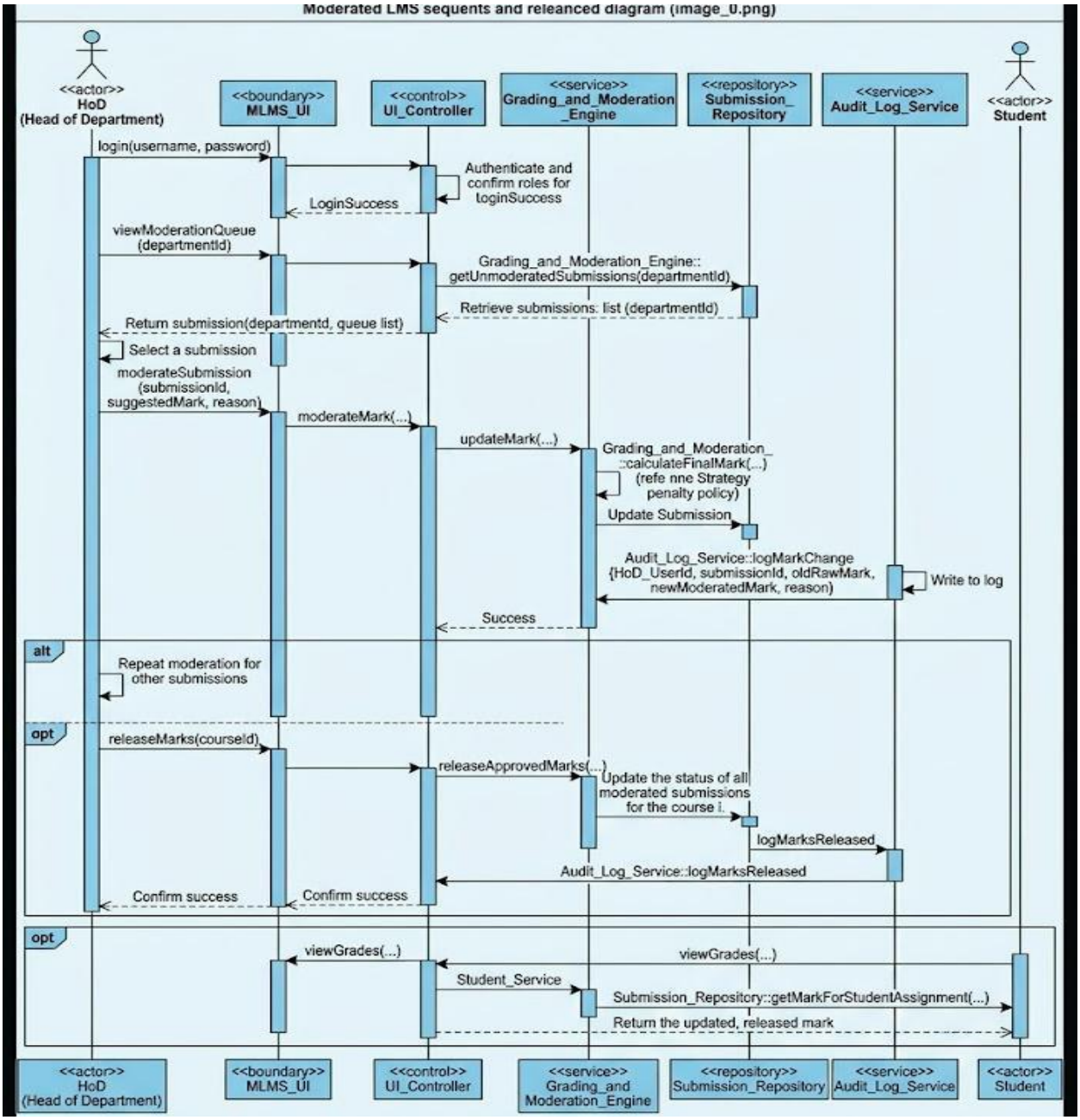
ii. **Activity Diagram**



iii. **Submit Assignment**

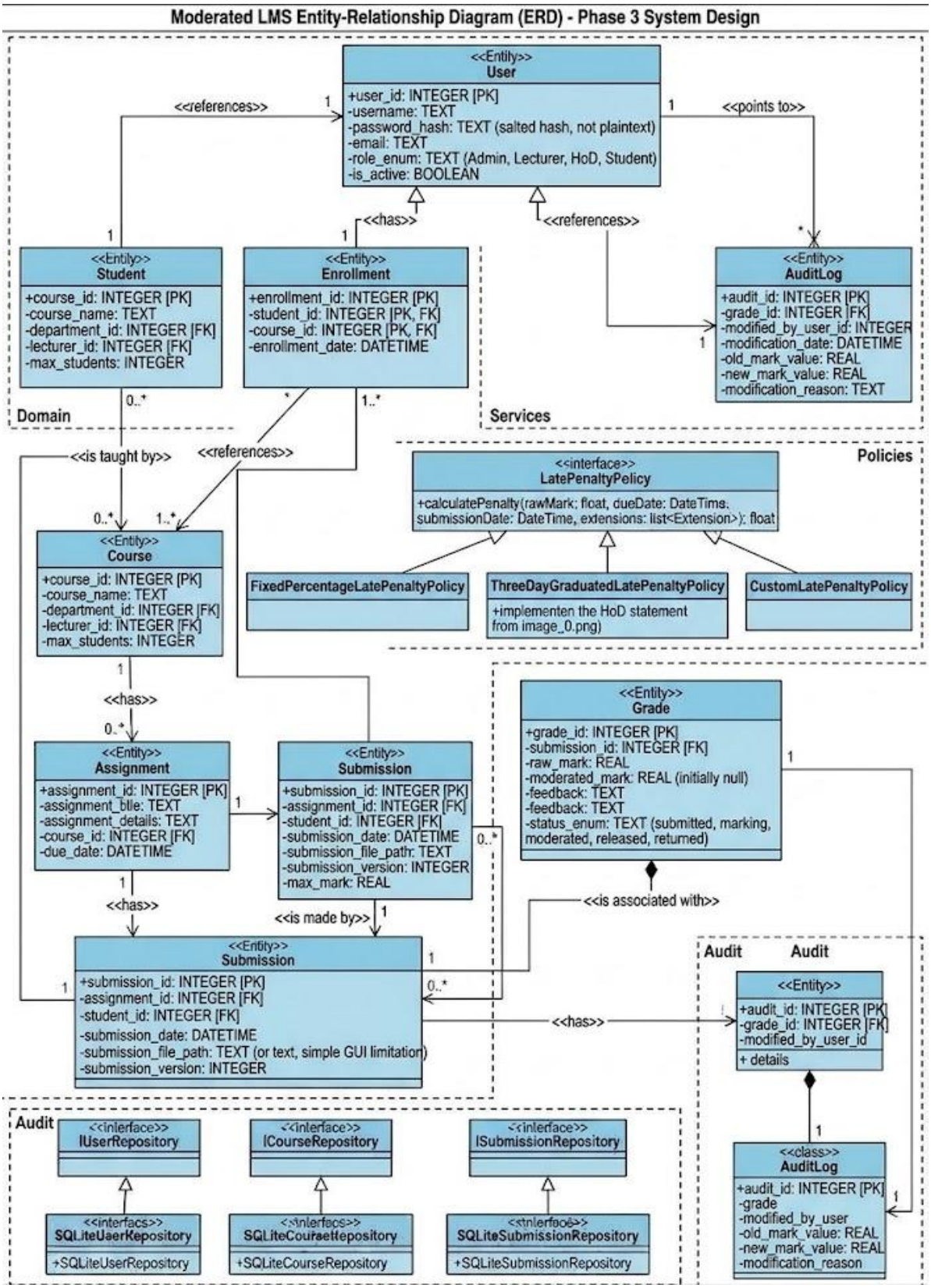


iv. Moderate Marks



Database Design

- ERD and Normal Form


Moderated LMS Entity-Relationship Diagram (ERD) - Phase 3 System Design
<<Entity>>
User
+user_id: INTEGER [PK]
-username: TEXT
-password_hash: TEXT (salted hash, not plaintext)
-email: TEXT
-role_enum: TEXT (Admin, Lecturer, HoD, Student)
-is_active: BOOLEAN
<<references>>
<<points to>>
<<has>>
<<references>>
<<Entity>>
Student
+course_id: INTEGER [PK]
-course_name: TEXT
-department_id: INTEGER [FK]
-lecturer_id: INTEGER [FK]
-max_students: INTEGER
<<Entity>>
Enrollment
+enrollment_id: INTEGER [PK]
-student_id: INTEGER [PK, FK]
-course_id: INTEGER [PK, FK]
-enrollment_date: DATETIME
<<Entity>>
AuditLog
+audit_id: INTEGER [PK]
-grade_id: INTEGER [FK]
-modified_by_user_id: INTEGER
-modification_date: DATETIME
-old_mark_value: REAL
-new_mark_value: REAL
-modification_reason: TEXT
Domain
Services
<<is taught by>>
<<references>>
Policies
<<interface>>
LatePenaltyPolicy
+calculatePenalty(rawMark: float, dueDate: DateTime, submissionDate: DateTime, extensions: list<Extension>): float
FixedPercentageLatePenaltyPolicy
ThreeDayGraduatedLatePenaltyPolicy
+implementen the HoD statement from image_0.png)
CustomLatePenaltyPolicy
<<Entity>>
Course
+course_id: INTEGER [PK]
-course_name: TEXT
-department_id: INTEGER [FK]
-lecturer_id: INTEGER [FK]
-max_students: INTEGER
<<has>>
<<Entity>>
Grade
+grade_id: INTEGER [PK]
-submission_id: INTEGER [FK]
-raw_mark: REAL
-moderated_mark: REAL (initially null)
-feedback: TEXT
-feedback: TEXT
-status_enum: TEXT (submitted, marking, moderated, released, returned)
<<Entity>>
Assignment
+assignment_id: INTEGER [PK]
-assignment_title: TEXT
-assignment_details: TEXT
-course_id: INTEGER [FK]
-due_date: DATETIME
<<Entity>>
Submission
+submission_id: INTEGER [PK]
-assignment_id: INTEGER [FK]
-student_id: INTEGER [FK]
-submission_date: DATETIME
-submission_file_path: TEXT
-submission_version: INTEGER
-max_mark: REAL
<<has>>
<<is made by>>
<<is associated with>>
<<Entity>>
Submission
+submission_id: INTEGER [PK]
-assignment_id: INTEGER [FK]
-student_id: INTEGER [FK]
-submission_date: DATETIME
-submission_file_path: TEXT (or text, simple GUI limitation)
-submission_version: INTEGER
Audit
Audit
<<Entity>>
+audit_id: INTEGER [PK]
-grade_id: INTEGER [FK]
-modified_by_user_id
+ details
<<has>>
Audit
<<interface>>
IUserRepository
<<interface>>
ICourseRepository
<<interface>>
ISubmissionRepository
<<interface>>
SQLiteUserRepository
+SQLiteUserRepository
<<interface>>
SQLiteCourseRepository
+SQLiteCourseRepository
<<interface>>
SQLiteSubmissionRepository
+SQLiteSubmissionRepository
<<class>>
AuditLog
+audit_id: INTEGER [PK]
-grade
-modified_by_user
-old_mark_value: REAL
-new_mark_value: REAL
-modification_reason

- **1NF (First Normal Form):** All tables contain atomic values, with repeating groups removed (e.g., separating assignment submissions and audit records into dedicated child tables).
- **2NF (Second Normal Form):** All non-key attributes are fully functionally dependent on the primary key (no partial dependencies on composite keys).
- **3NF (Third Normal Form):** There are no transitive dependencies; non-key attributes depend *only* on the primary key (e.g., department details reside strictly in a Departments table rather than being duplicated across users or courses).

Data Dictionary

Table	Field	Type	Constraints / purpose
Department	Department_id	INTEGER	PK; unique department identifier
	name	TEXT	Required, unique department name
UserAccount	user_id	INTEGER	PK; unique user identifier
	username	TEXT	Required, unique login name
	email	TEXT	Required, unique login name
	password_hash	TEXT	Required secure password hash
	role	TEXT	Required; Student, Lecturer, Head of Department, Administrator
	department_id	INTEGER	FK; user's department, if applicable
Course	course_id	INTEGER	PK; Unique identifier for a course.
	name	TEXT	Required course name
	department_id	INTEGER	FK to Department
Enrollment	student_id	INTEGER	PK component; FK to UserAccount
	course_id	INTEGER	PK component; FK to Course
	enrolled_at	TEXT	Enrollment timestamp
Assignments	assignment_id	INTEGER	PK; Unique identifier for coursework.
	course_id	INTEGER	FK to Course
	title	TEXT	Assignment title.
	due_date	DATETIME	Original submission deadline.
	max_mark	REAL	Maximum available points.

	allow_late	BOOLEAN	Flag indicating if late submissions are permitted
Submissions	submission_id	INTEGER	PK; Unique identifier for a student submission.
	assignment_id	INTEGER	FK; Target assignment.
	student_id	INTEGER	FK; Student who submitted.
	raw_mark	REAL	Initial mark awarded by the lecturer.
	moderated_mark	REAL	Final moderated mark set by the Head of Departments.
	status	TEXT	Lifecycle state
	submitted_at	DATETIME	Timestamp of file upload.
AuditLogs	log_id	INTEGER	PK; Unique log entry identifier.
	submission_id	INTEGER	FK; Submission affected by the change.
	performed_by	INTEGER	FK; User ID who triggered the change
	timestamp	DATETIME	Exact time of modification
	old_value	TEXT	Previous mark value before change
	new_value	TEXT	New mark value after change
	reason	TEXT	Mandatory justification for modification

- **SQL Schema**

```
-- Enable foreign key support in SQLite

PRAGMA foreign_keys = ON;


CREATE TABLE Departments (

   department_id INTEGER PRIMARY KEY AUTOINCREMENT,

   department_name TEXT NOT NULL UNIQUE

);
```

```
CREATE TABLE Users (

   user_id INTEGER PRIMARY KEY AUTOINCREMENT,

   username TEXT NOT NULL UNIQUE,

   email TEXT NOT NULL,

   password_hash TEXT NOT NULL,

   role TEXT NOT NULL CHECK(role IN ('Student', 'Lecturer', 'HeadOfDepartment',
'Administrator')),

   department_id INTEGER,

   FOREIGN KEY (department_id) REFERENCES Departments(department_id) ON DELETE SET
NULL

);



CREATE TABLE Courses (

   course_id INTEGER PRIMARY KEY AUTOINCREMENT,

   course_name TEXT NOT NULL,

   department_id INTEGER NOT NULL,

   FOREIGN KEY (department_id) REFERENCES Departments(department_id) ON DELETE
CASCADE

);



CREATE TABLE Enrollments (

   enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,

   course_id INTEGER NOT NULL,

   student_id INTEGER NOT NULL,
```

```
    FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON DELETE CASCADE,
    FOREIGN KEY (student_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    UNIQUE(course_id, student_id)
);

CREATE TABLE Assignments (
    assignment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    due_date DATETIME NOT NULL,
    max_mark REAL NOT NULL CHECK(max_mark > 0),
    allow_late BOOLEAN NOT NULL DEFAULT 1,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON DELETE CASCADE
);

CREATE TABLE Submissions (
    submission_id INTEGER PRIMARY KEY AUTOINCREMENT,
    assignment_id INTEGER NOT NULL,
    student_id INTEGER NOT NULL,
    raw_mark REAL CHECK(raw_mark >= 0),
    moderated_mark REAL CHECK(moderated_mark >= 0),
    status TEXT NOT NULL DEFAULT 'Submitted' CHECK(status IN ('Submitted', 'Late', 'Marked',
'Moderated', 'Released')),
```

```
    submitted_at DATETIME NOT NULL,

    FOREIGN KEY (assignment_id) REFERENCES Assignments(assignment_id) ON DELETE
CASCADE,

    FOREIGN KEY (student_id) REFERENCES Users(user_id) ON DELETE CASCADE

);


CREATE TABLE AuditLogs (

    log_id INTEGER PRIMARY KEY AUTOINCREMENT,

    submission_id INTEGER NOT NULL,

    performed_by INTEGER NOT NULL,

    timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    old_value TEXT,

    new_value TEXT,

    reason TEXT NOT NULL,

    FOREIGN KEY (submission_id) REFERENCES Submissions(submission_id) ON DELETE
CASCADE,

    FOREIGN KEY (performed_by) REFERENCES Users(user_id) ON DELETE RESTRICT

);
```

- **Mapping Classes to Tables & Justification of Differences**

1. Inheritance Mapping (User Hierarchy vs. Users Table):
    - *Mapping:* In the Class Diagram, User is an abstract class with four subclasses (Student, Lecturer, HeadOfDepartment, Administrator). In the relational schema, these are collapsed into a Single Table Inheritance model within the Users table, differentiated by a role enum column and an optional department_id foreign key.
    - *Justification:* Collapsing subclasses into one table avoids complex multi-table JOIN operations during authentication and keeps user profile management unified, especially since all users share core credentials (username, password_hash, email).
2. Handling Lists and Collections (e.g., assignments: list<Assignment> in Course):
    - *Mapping:* In C++, classes contain container objects like std::vector<Assignment>. In the database, these object references are converted into foreign key relationships (course_id stored inside the Assignments table).
    - *Justification:* Relational databases cannot store object collections natively in single cells without violating First Normal Form (1NF). Using foreign keys maintains strict normalization while repositories easily hydrate C++ object collections on demand.
3. Audit Logging Separation (AuditRecord):
    - *Mapping:* The AuditRecord class maps directly to the AuditLogs table. However, fields like performedByUserId are enforced via relational foreign keys (performed_by referencing Users(user_id)) rather than raw object pointers.
    - *Justification:* This ensures referential integrity; a mark change cannot be logged against a non-existent user, and database constraints prevent accidental deletion of user audit trails (ON DELETE RESTRICT).